PLSC 40601

Week 4: Representation learning for causal inference.

Molly Offer-Westort

Department of Political Science,
University of Chicago

Spring 2026

Representations.

From prediction to causal prediction

- In ordinary supervised learning, we observe an input vector X with p components and a target Y. We fit $\hat{f}(X)$ to predict Y.

- In causal inference, we often want

$$\mu_1(X) = \mathrm{E}\left[Y(1) \mid X\right], \qquad \mu_0(X) = E[Y(0) \mid X]$$

  and

$$\tau(X) = \mu_1(X) - \mu_0(X).$$

- For each unit, one counterfactual outcome is always missing.

- This is a *transfer learning* problem: we want to infer something about the missing potential outcome from the observed data.

Why not stay linear?

- If treatment effects vary in complicated ways, linear main-effects models can be too rigid.
- We can add interactions or basis expansions, but hand-crafting them gets expensive quickly.

Additive models

- An additive model writes the regression function as a sum of one-variable components:

$$f(X) = \sum_{j=1}^{p} h_j(X_j).$$

- Each covariate enters flexibly through its own function $h_j(\cdot)$ rather than through a single linear coefficient.

- Example:

$$\mathrm{E}[Y \mid X] = h_1(\text{age}) + h_2(\text{income}) + h_3(\text{prior turnout}).$$

- This is more flexible than linear regression, but it still does not include interactions unless we add them explicitly.

Hastie et al. (2009); Bishop (2006)

Basis expansions

- A standard way to allow nonlinear regression is to replace raw inputs with transformed features $h_1(X), \ldots, h_M(X)$.

- Then fit

$$f(X) = \sum_{m=1}^{M} \beta_m h_m(X).$$

- If the basis functions are fixed in advance, we estimate β just as in linear regression by minimizing

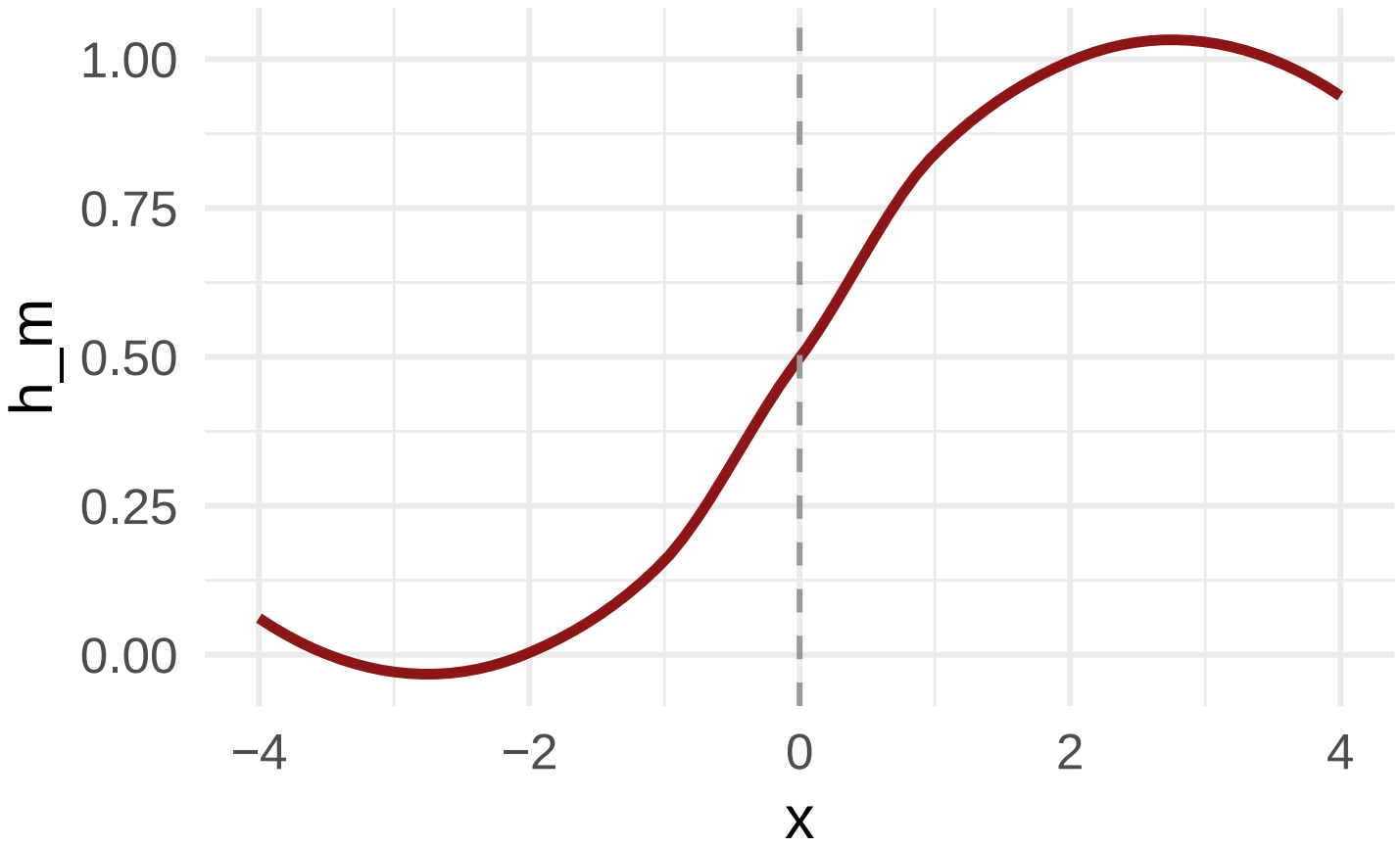
$$\mathrm{RSS}(\beta) = \sum_{i=1}^{N} (y_i - f(X_i))^2 .$$

- Examples include polynomial terms, spline bases, and localized functions such as sigmoids or radial basis functions.

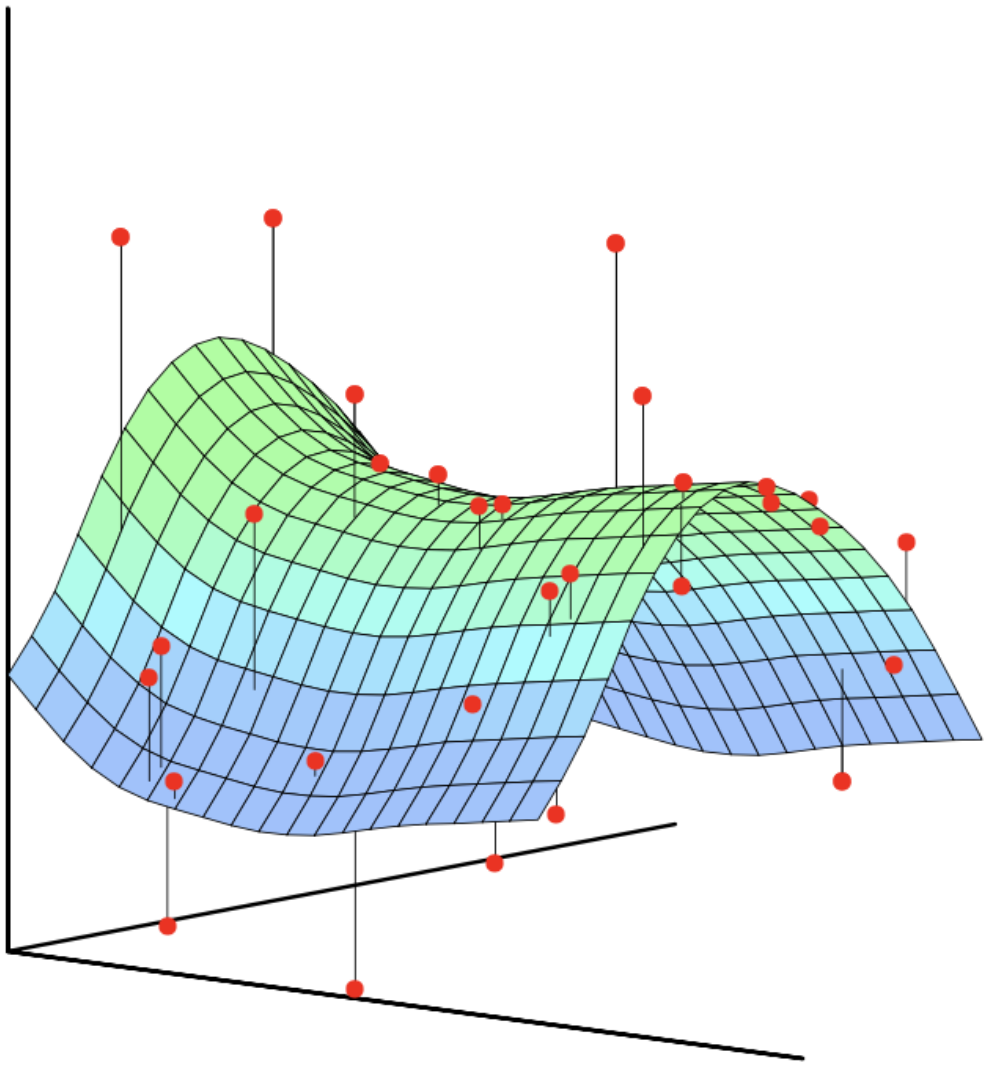
A sigmoid basis function

$$h_m(x) = \frac{1}{1 + \exp\{-s_m(x - \mu_m)\}}$$

```
> x <- seq(-4, 4, length.out = 200)
> mu_m <- 0
> s_m <- 2
> h_m <- 1 / (1 + exp(-s_m * (x - mu_m)))
```



Basis expansion example



Hastie et al. (2009)

Project pursuit regression.

Projection pursuit regression

- Let ω_m, for $m = 1, 2, \ldots, M$, be unit p-vectors of unknown parameters, and define derived features

$$V_m = \omega_m^\top X.$$

- The projection pursuit regression model is

$$f(X) = \sum_{m=1}^{M} g_m(\omega_m^\top X).$$

- Given training data $\{(x_i, y_i)\}_{i=1}^N$, we seek parameters

$$\theta = \{\omega_m, g_m\}_{m=1}^M$$

by minimizing the criterion function

$$\sum_{i=1}^{N} \left[y_i - \sum_{m=1}^{M} g_m(\omega_m^\top x_i) \right]^2 .$$

Projection pursuit regression

- The projection pursuit regression model is

$$f(X) = \sum_{m=1}^{M} g_m(\omega_m^\top X).$$

- This is an additive model, but in learned projections rather than the original inputs.

- Unlike fixed basis expansions, both the projections ω_m and the functions g_m are learned from the data.

Fitting one ridge term

- Start with the simplest case, $M = 1$, so

$$f(X) = g(\omega^\top X).$$

- For any fixed direction ω, form the projected feature

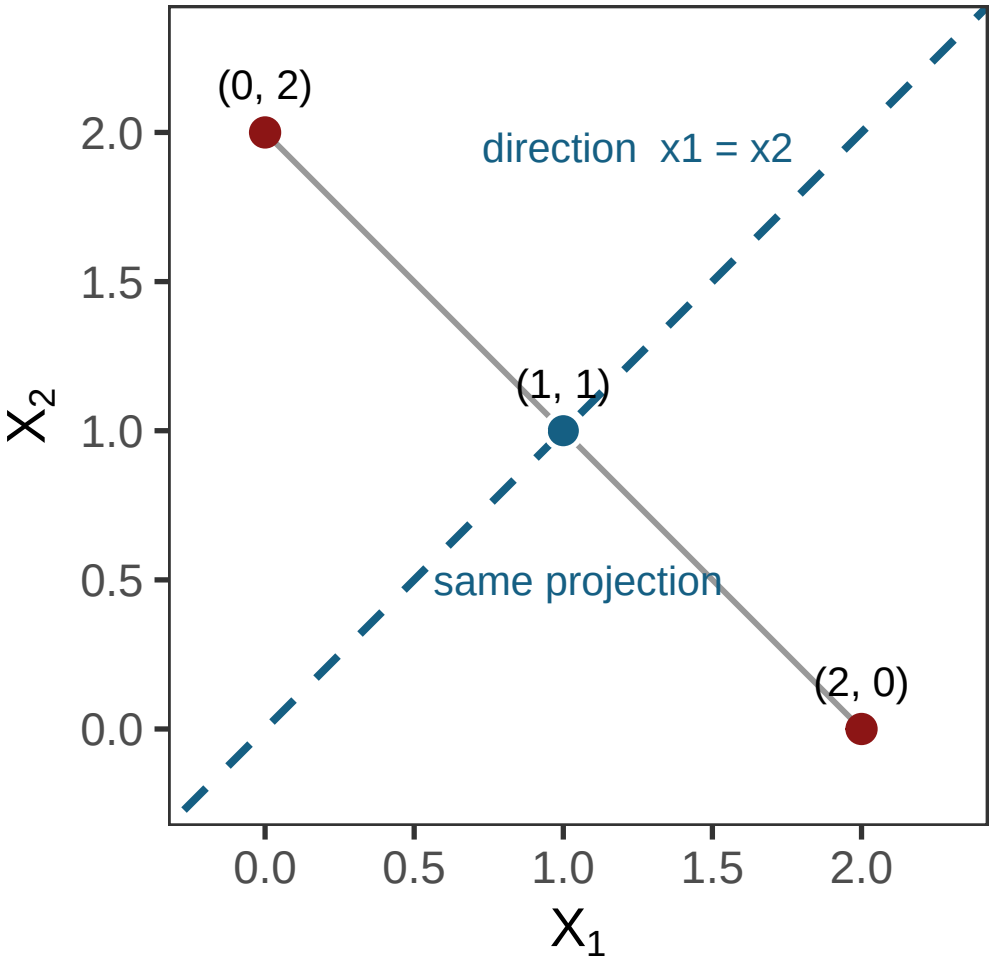
$$z_i = \omega^\top x_i.$$

- Then estimate g by a one-dimensional smoother or regression of y_i on z_i.
- Next search over ω to reduce residual sum of squares.
- So even in the one-term case, PPR alternates between **projecting** the data and **smoothing** along that projection.

Toy example: what a direction does

- Suppose $X = (X_1, X_2)$ and choose $\omega = (1,1)/\sqrt{2}$.
  - Why? If you want the direction that gives equal weight to X_1 and X_2, the natural vector is $(1,1)$. But in PPR and related models we usually normalize direction vectors to have length 1, so we divide by its norm:
    $$\|(1,1)\| = \sqrt{1^2 + 1^2} = \sqrt{2},$$
    which gives
    $$\omega = \frac{(1,1)}{\sqrt{2}}.$$
  - That does two things:
    - It makes ω a unit vector, so it encodes direction only.
    - Avoids ambiguity: $(1,1)$, $(2,2)$, and $(10,10)$ all point the same way but would give different projection magnitudes unless you normalize.
- Then $\omega^\top X = (X_1 + X_2)/\sqrt{2}$, so the ridge term looks at the diagonal combination of the two covariates.
- The points $(2,0)$, $(1,1)$, and $(0,2)$ all have $\omega^\top X = \sqrt{2}$.
- So $g(\omega^\top X)$ gives those points the same contribution.

Toy example: what a direction does



Toy example: estimate g from z_i

- Keep the same direction $\omega = (1, 1)/\sqrt{2}$.

- For each observation, compute the scalar projection

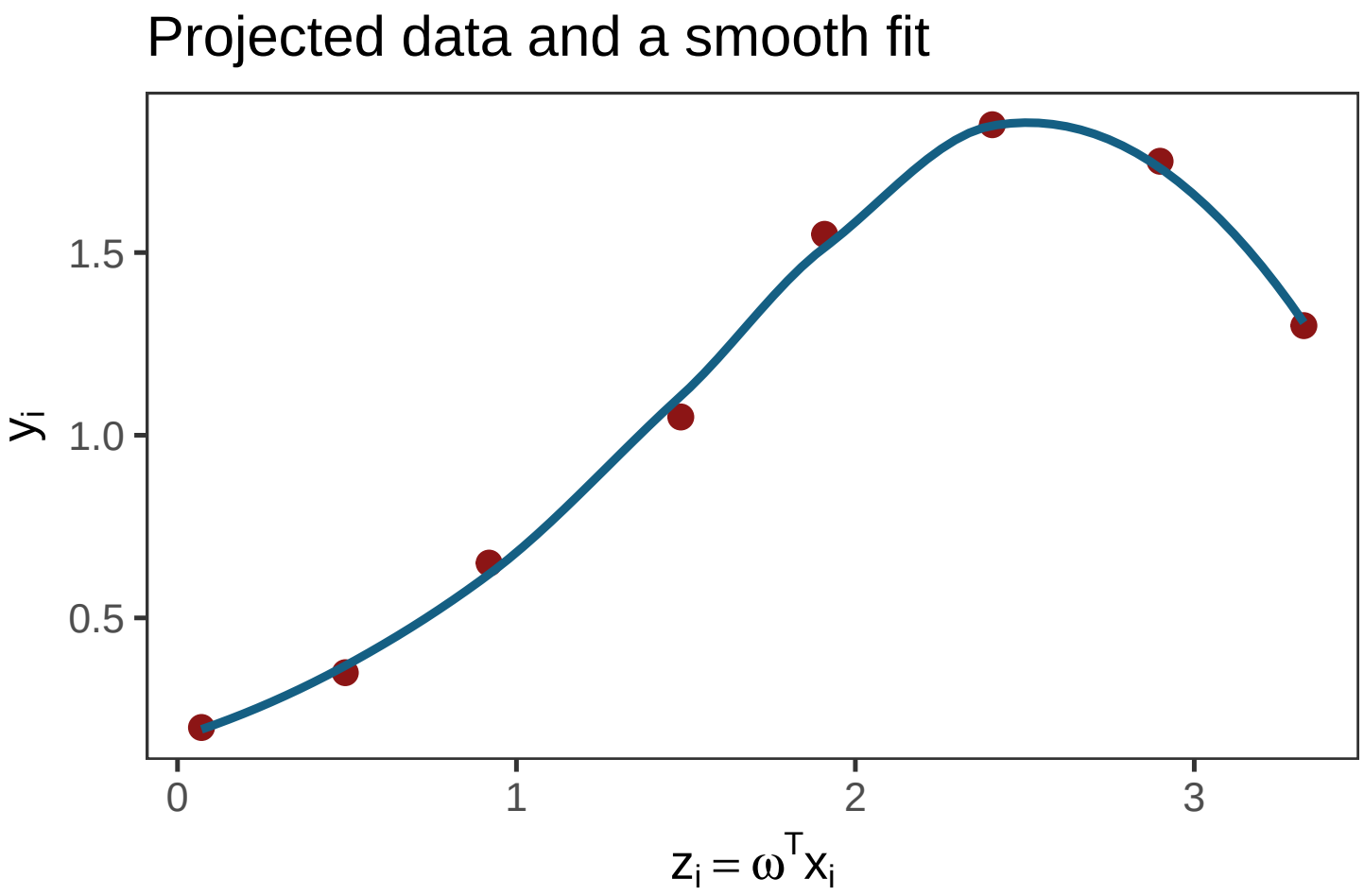
$$z_i = \omega^\top x_i.$$

- The three points on the previous slide only showed what it means to share the same projection.

- To estimate the ridge function g, we need several observations with different values of z_i.

- Then we can fit a one-dimensional smoother or regression of y_i on z_i.

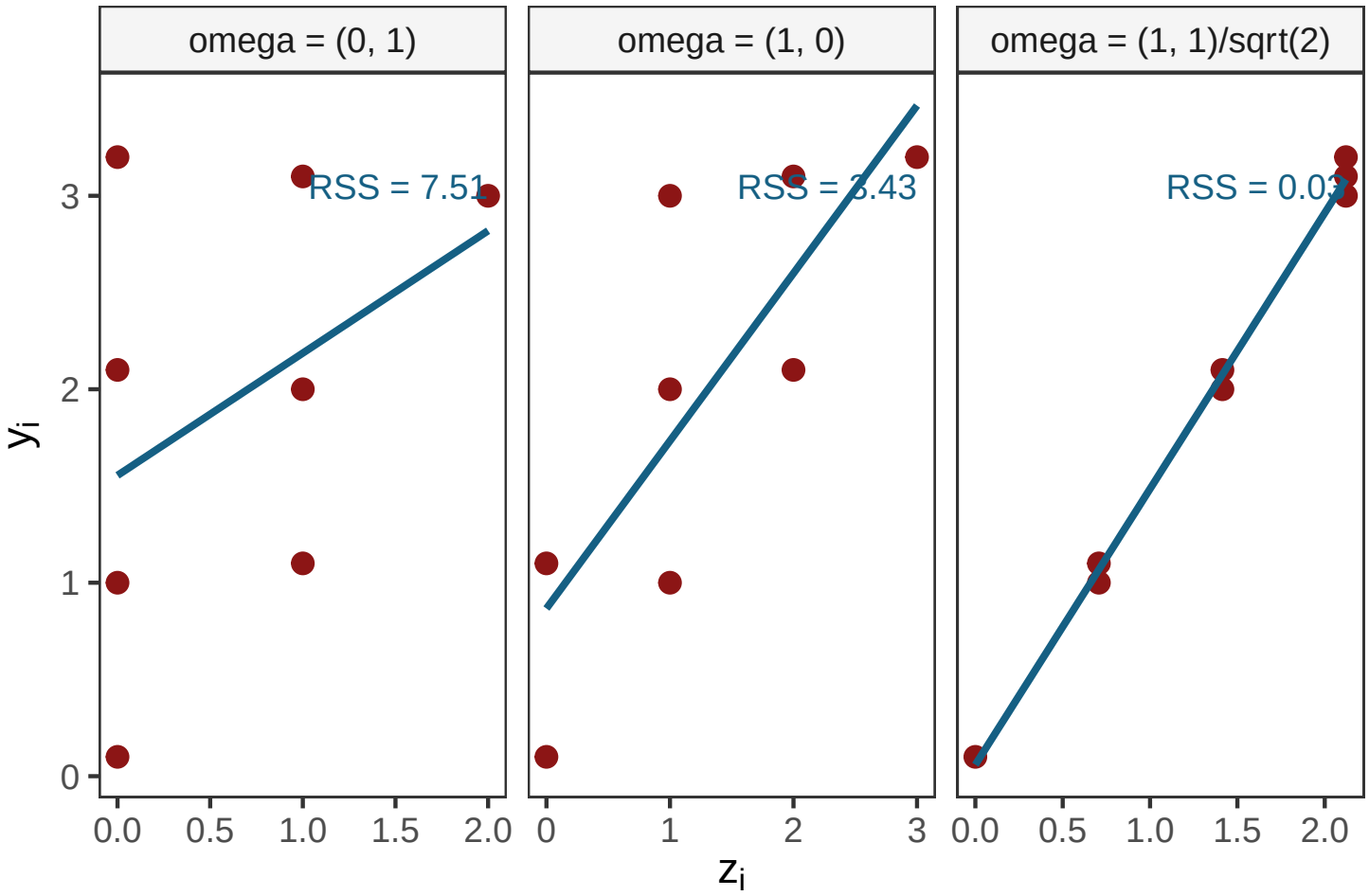
Toy example: estimate g from z_i



Toy example: search over ω

- Now try several candidate directions ω.
- For each one, compute
$$z_i = \omega^\top x_i,$$
fit a one-dimensional regression of y_i on z_i, and record the residual sum of squares.
- The best direction is the one with the smallest RSS.
- In this toy data, the diagonal direction does best because y depends mostly on $x_1 + x_2$.

Toy example: search over ω



Backfitting when $M > 1$

- With several ridge terms, hold all terms except m fixed and update one component at a time.

- Define the partial residual for term m as

$$r_i^{(m)} = y_i - \sum_{k \neq m} \hat{g}_k(\omega_k^\top x_i).$$

- Then update (g_m, ω_m) by solving

$$\min_{g_m, \omega_m} \sum_{i=1}^{N} \left[r_i^{(m)} - g_m(\omega_m^\top x_i) \right]^2.$$

- Equivalently, we replace the old mth ridge function with a new one while treating the others as fixed.

- Cycling through $m = 1, \ldots, M$ gives a Gauss-Seidel or **backfitting** algorithm.

Practical fitting of PPR

- In practice, the algorithm alternates through the M terms repeatedly until the fit stops improving much.

- A common implementation is **forward stagewise**: add one new ridge term at a time, then refine the existing terms.

- The number of terms M is usually chosen by validation or cross-validation rather than estimated in one joint optimization step.

- This is computationally heavier, and often less stable, than fitting ordinary linear or additive models.

- That computational burden is one reason neural networks largely displaced PPR in many applications.

Interpreting PPR

- A single term $g_m(\omega_m^\top X)$ is called a **ridge function**.
- The projection
$$V_m = \omega_m^\top X$$
gives the view of the covariates in the direction defined by ω_m.
- Then g_m is just a one-dimensional smooth function of that projected feature.
- Geometrically, $g_m(\omega_m^\top X)$ is constant on hyperplanes orthogonal to ω_m.
- So PPR searches for useful directions in covariate space and then fits flexible univariate functions along those directions.

PPR is very flexible

- If M is sufficiently large, the PPR model can approximate any continuous function on $\mathbb{R}^p$ arbitrarily well: it is a **universal approximator**.

- But: once several projection terms are combined, interpretation becomes difficult; covariates can enter the fit through multiple projections, and the projections themselves are not necessarily easy to understand.

- So PPR is usually better viewed as a flexible predictive model than as a simple, transparent description of the data-generating process.

- One important exception is the case $M = 1$, sometimes called the **single-index model** in econometrics.

- That model is more general than linear regression, while still being relatively interpretable.

Neural nets.

A simple feedforward network

- A one-hidden-layer feedforward network maps an input vector

$$X = (X_1, \ldots, X_p)^\top$$

  to a prediction.

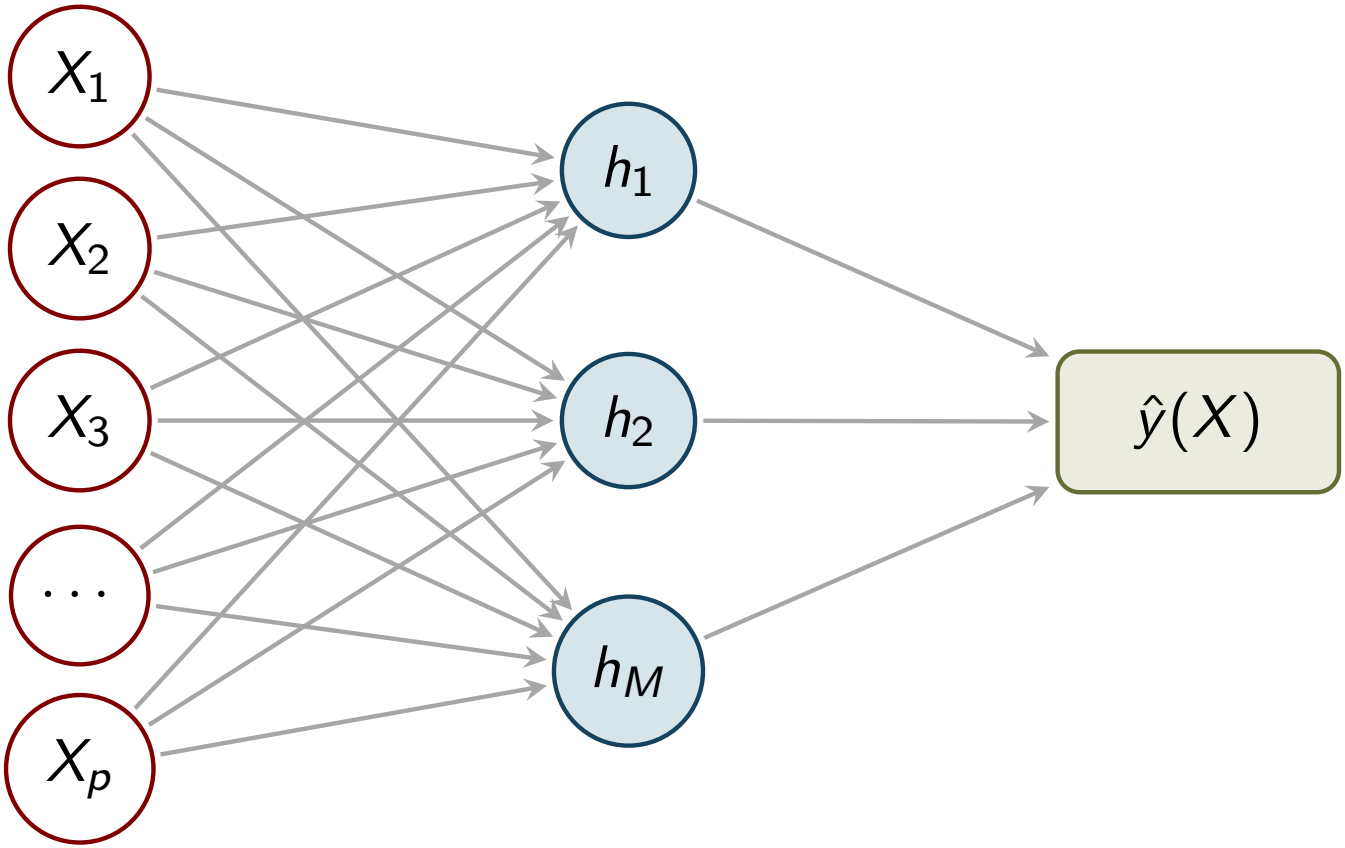
- Between the inputs and the prediction sits a layer of M hidden units.

- In a fully connected network, each hidden unit depends on all input coordinates, and the output depends on all hidden units.

- Information flows only forward: inputs $\to$ hidden layer $\to$ output.

- We focus first on the one-output regression case; classification mainly changes the last layer.

One hidden layer, schematically



observed covariates learned hidden features prediction

The hidden layer provides learned features that are then combined into a prediction.

One hidden layer in notation

- For hidden unit m, form the pre-activation

$$z_m(X) = \alpha_{0m} + \alpha_m^\top X, \qquad m = 1, \ldots, M,$$

- Then apply the hidden-unit activation

$$h_m(X) = \sigma(z_m(X)).$$

- In the one-output regression case, the prediction is

$$\hat{y}(X) = \beta_0 + \sum_{m=1}^{M} \beta_m h_m(X).$$

- We will use z_m for hidden pre-activations, h_m for hidden-unit outputs, and $\hat{y}$ for the prediction.

Activation functions

- The key nonlinear ingredient is the hidden-unit activation $\sigma(\cdot)$.
- A classic choice is the sigmoid function

$$\sigma(v) = \frac{1}{1 + e^{-v}}.$$

- What if $\sigma(\cdot)$ were linear?
- In a regression neural net, we usually do not apply any extra nonlinear transformation at the end, so the prediction is the weighted sum of hidden units.
- In multi-class classification, we can replace the scalar output with logits $t_k(X)$ and softmax probabilities

$$\hat{p}_k(X) = \frac{e^{t_k(X)}}{\sum_{\ell=1}^{K} e^{t_\ell(X)}}.$$

Hidden units as learned basis functions

- In the one-output regression case, the network has the form

$$h_m(X) = \sigma(\alpha_{0m} + \alpha_m^\top X), \qquad \hat{y}(X) = \beta_0 + \sum_{m=1}^{M} \beta_m h_m(X).$$

- This has the same outer form as a basis expansion: a linear combination of transformed features.

- The difference is that the features $h_m(X)$ are not fixed in advance.

- The data determine both the directions α_m and the nonlinear transformations through the fitted weights.

- So hidden units are best viewed as **learned basis functions** or **learned features**.

One hidden layer as constrained PPR

- We can rewrite the network as:

$$\hat{y}(X) = \beta_0 + \sum_{m=1}^{M} \beta_m \sigma(\alpha_{0m} + \alpha_m^\top X).$$

- This has the same overall ridge-function structure as PPR:

$$f(X) = \sum_{m=1}^{M} g_m(\omega_m^\top X).$$

- We can identify $\omega_m = \alpha_m, \qquad g_m(v) = \beta_m \sigma(\alpha_{0m} + v).$
- PPR estimates a general smooth ridge function g_m, while the neural net restricts each ridge term to the activation family $\sigma(\cdot)$.
- Because that family is more constrained, neural nets may need more hidden units M than PPR would need ridge terms.

Fitting a neural network

- Collect all unknown weights and intercepts into a parameter vector θ.

- With p inputs and M hidden units, the one-output model has

$$(p+1)M + (M+1) = (p+2)M + 1$$

  parameters.

- For training data $(x_i, y_i)_{i=1}^N$, we choose θ by minimizing a loss function.

- In the regression case, a standard choice is squared error loss:

$$L(\theta) = \sum_{i=1}^{N} [y_i - \hat{y}(x_i)]^2 .$$

- For classification, the analogous choice is cross-entropy loss.

- Find a high-dimensional parameter vector that makes the fitted

Optimization and backpropagation

- Unlike PPR backfitting, neural-network training updates all weights jointly.
- The loss is generally nonconvex, so we rely on gradient-based optimization rather than a closed-form solution.
- A forward pass computes the hidden pre-activations $z_m(x_i)$, hidden outputs $h_m(x_i)$, and predictions $\hat{y}(x_i)$.
- A backward pass applies the chain rule to compute how each weight affects the loss.
- That backward recursion is **backpropagation**.

Backpropagation for squared error

- For one observation i, write the contribution to the loss as

$$\ell_i(\theta) = [y_i - \hat{y}(x_i)]^2 .$$

- Define

$$z_{im} = \alpha_{0m} + \alpha_m^\top x_i, \qquad h_{im} = \sigma(z_{im}), \qquad \hat{y}_i = \beta_0 + \sum_{m=1}^{M} \beta_m h_{im}.$$

- Then the output-layer gradient is

$$\frac{\partial \ell_i}{\partial \beta_m} = -2\,(y_i - \hat{y}_i)\, h_{im}.$$

- And the input-to-hidden gradient is

$$\frac{\partial \ell_i}{\partial \alpha_{jm}} = -2\,(y_i - \hat{y}_i)\, \beta_m \sigma'(z_{im}) x_{ij}.$$

Error signals and gradient updates

- Define the output error signal

$$\delta_i = -2\left(y_i - \hat{y}_i\right).$$

- Define the hidden-unit error signal

$$\eta_{im} = \delta_i \beta_m \sigma'(z_{im}).$$

- Then the gradients factor as

$$\frac{\partial \ell_i}{\partial \beta_m} = \delta_i h_{im}, \qquad \frac{\partial \ell_i}{\partial \alpha_{jm}} = \eta_{im} x_{ij}.$$

- A batch gradient update takes the form

$$\beta_m^{(r+1)} = \beta_m^{(r)} - \gamma_r \sum_{i=1}^{N} \frac{\partial \ell_i}{\partial \beta_m}, \qquad \alpha_{jm}^{(r+1)} = \alpha_{jm}^{(r)} - \gamma_r \sum_{i=1}^{N} \frac{\partial \ell_i}{\partial \alpha_{jm}}.$$

Putting it together for representation learning.

Representation learning

- Write $\phi(X)$ for a learned representation of the covariates.
- Then predict outcomes using $\phi(X)$ rather than raw X.
- Intuition: a good representation keeps the signal that matters for outcomes and drops noise that does not.
- Bridging to Johansson et al. (2016); Shalit et al. (2017)

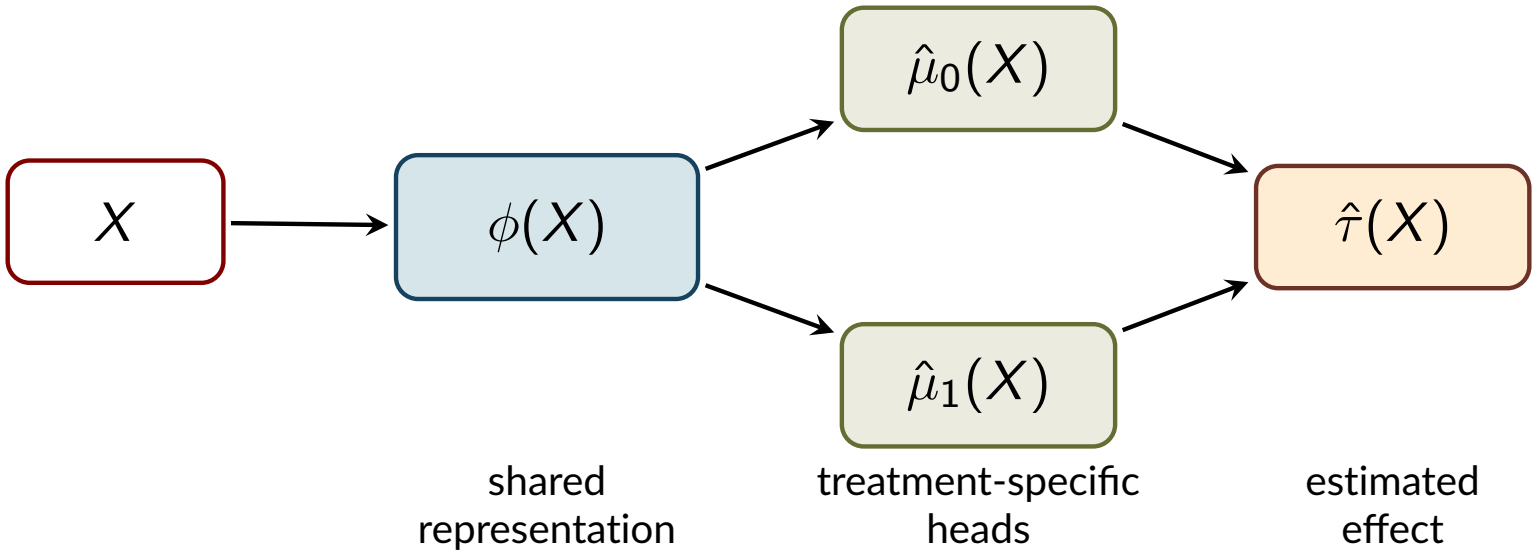
Shared trunk, separate heads

- One natural architecture is

$$\hat{\mu}_0(X) = h_0(\phi(X)), \qquad \hat{\mu}_1(X) = h_1(\phi(X)).$$

- The shared representation $\phi(X)$ pools structure across treatment arms.
- The separate heads let treated and control response surfaces differ.
- Core idea of TARNet-style models (Treatment-Agnostic Representation Network).

Shared representation, separate heads



Why this is attractive for causal heterogeneity

- Many covariates may matter, but not all in the same way for treated and control units.
- Sharing lower-level features can stabilize estimation when each arm is limited.
- The target is heterogeneity.

What neural nets do not solve

- A flexible architecture does not identify causal effects by itself.
- The papers still rely on unconfoundedness and adequate overlap.
- If treated and control units live in different regions of covariate space, the model is still being asked to extrapolate.

References I

Bishop, C. M. (2006). *Pattern Recognition and Machine Learning (Information Science and Statistics)*. Springer-Verlag, New York.

Hastie, T., Tibshirani, R., and Friedman, J. H. (2009). *The elements of statistical learning: data mining, inference, and prediction*, volume 2. Springer.

Johansson, F., Shalit, U., and Sontag, D. (2016). Learning representations for counterfactual inference. In Balcan, M. F. and Weinberger, K. Q., editors, *Proceedings of the 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 3020–3029.

Shalit, U., Johansson, F. D., and Sontag, D. (2017). Estimating individual treatment effect: Generalization bounds and algorithms. In Precup, D. and Teh, Y. W., editors, *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 3076–3085.